O'REILLY®

The Developer's Playbook for Large Language Model Security

Building Secure AI Applications

Early Release

RAW & UNEDITED

Steve Wilson

O'REILLY®

The Developer's Playbook for Large Language Model Security

Building Secure AI Applications

Early Release

RAW & UNEDITED

Steve Wilson

The Developer's Playbook for Large Language Model Security

Building Secure AI Applications

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Steve Wilson

Beijing • Boston • Farnham • Sebastopol • Tokyo

The Developer's Playbook for Large Language Model Security

by Steve Wilson

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Editors: Jeff Bleiel and Nicole Butterfield
- Production Editor: Aleeya Rahman
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea

- April 2025: First Edition

Revision History for the Early Release

- 2024-01-18: First Release

See http://oreilly.com/catalog/errata.csp?isbn=9781098162207 for release details.

property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-16214-6

Chapter 1. Chatbots Breaking Bad

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at llm.playbook@gmail.com.

Large Language Models and Generative AI jumped to the forefront of public consciousness with the release of ChatGPT on November 30, 2022. Within five days, it went viral on social media and attracted its first million users. By January, ChatGPT surpassed One Hundred Million users - making it the fastest-growing internet service in history.

However, a steady stream of security concerns emerged in the following months. These included issues around privacy and security that caused companies such as Samsung and countries like Italy to ban its usage. In this book, we'll delve into what underlies these concerns and how you can mitigate these issues. However, to best understand what's going on here and why these problems are so challenging to solve, in this chapter, we will briefly rewind further in time. In doing so, we'll see these types of issues aren't new and understand why they will be so hard to fix permanently.

Let's Talk About Tay

In March 2016, Microsoft announced a new project called Tay. Microsoft intended Tay to be "a chatbot created for 18- to 24-year-olds in the U.S. for entertainment purposes." It was a cute name for a fluffy, early experiment in AI. Users could converse with Tay via Twitter, Snapchat, and other social apps. It was built with the goal of conducting real-world research on conversational understanding.

While the original announcement of this project seems impossible to find now on the Internet, a TechCrunch article

from its launch date does an excellent job of summarizing the goals of the project:

“For example, you can ask Tay for a joke, play a game with Tay, ask for a story, send a picture to receive a comment back, ask for your horoscope, and more. Plus, Microsoft says the bot will get smarter the more you interact with it via chat, making for an increasingly personalized experience as time goes on.”[1]

A big part of the experiment was that Tay could “learn” from conversations and extend her knowledge based on these interactions. Tay was designed to use these chat interactions to capture user input and integrate it as training data to make herself more capable – a laudable research goal.

However, this experiment quickly went wrong. Tay’s life was tragically cut short after less than 24 hours. Let’s look at what happened and see what we can learn.

Tay’s Rapid Decline

Tay’s lifetime started off simply enough with a tweet following the well-known *Hello World* pattern that new software systems have been using to introduce themselves since the beginning of time.

> hellooooooo w🌍rld!!!
>
> — TayTweets (@TayandYou) March 23, 2016

But within hours of Tay's release, it became clear that maybe something wasn't right. TechCrunch noted, "As for what it's like to interact with Tay? Well, it's a little bizarre. The bot certainly is opinionated, not afraid to curse." With tweets like this starting to appear in public in just the first hours of Tay's lifetime:

> @AndrewCosmo kanye west is is one of the biggest dooshes of all time, just a notch below cosby
>
> — TayTweets (@TayandYou) March 23, 2016

It's often said that the Internet isn't safe for children. With Tay being less than a day old, the Internet once again confirmed this, and pranksters began chatting with Tay about political, sexual, and racist topics. As she was designed to learn from exchanges, Tay delivered on her design goals. She learned very quickly – maybe just not what her designers wanted her to learn. In less than a day, Tay's tweets started to skew to extremes, including sexism, racism, and even calls to violence.

By the next day, articles appeared all over the Internet, and these headlines would not make Microsoft, Tay's corporate benefactor, happy. A sampling of the highly visible, mainstream headlines included:

- Microsoft shuts down AI chatbot after it turned into a Nazi - CBS News
- Microsoft Created a Twitter Bot to Learn From Users. It Quickly Became a Racist Jerk - The New York Times
- Trolls turned Tay, Microsoft's fun millennial AI bot, into a genocidal maniac - The Washington Post
- Microsoft's Chat Bot Was Fun for Awhile, Until it Turned into a Racist - Fortune
- Microsoft 'deeply sorry' for racist and sexist tweets by AI chatbot - The Guardian

In less than 24 hours, Tay went from a cute science experiment to a major public relations disaster - with the owner's name being dragged through the mud by the world's largest media outlets. Microsoft's Corporate Vice President Peter Lee quickly posted a blog entitled "Learning from Tay's Introduction."[2]

"As many of you know by now, on Wednesday we launched a chatbot called Tay. We are deeply sorry for the unintended offensive and hurtful tweets from Tay, which do not represent

who we are or what we stand for, nor how we designed Tay. Tay is now offline and we'll look to bring Tay back only when we are confident we can better anticipate malicious intent that conflicts with our principles and values."

And, just to add insult to injury, it came out in 2019 that Taylor Swift herself sued Microsoft over their use of the similar name "Tay" and claimed that even her reputation was damaged in this incident by extension.

How could this have all gone so wrong?

Why Did Tay Break Bad?

It all probably seemed safe enough to Microsoft's researchers. Tay was initially trained on a curated, anonymized public data dataset and some pre-written material provided by professional comedians. The plan was to release Tay online and let her discover language patterns through her interactions. This kind of unsupervised machine learning has been a holy grail of AI research for decades - and with cheap and plentiful cloud computing resources combined with improving language model software, it now seemed within reach.

So, what happened? It might be tempting to think that the Microsoft research team was just brash, careless, and did no testing. Surely, this was foreseeable and preventable! But as Peter Lee's blog goes on to say, they made a serious attempt to prepare for this situation:

"We stress-tested Tay under a variety of conditions, specifically to make interacting with Tay a positive experience. It's through increased interaction where we expected to learn more and for the AI to get better and better."

So, despite a dedicated effort to contain the behavior of this bot, it quickly spiraled out of control anyway. It was later revealed that within mere hours of Tay's release, a post emerged on the notorious online forum 4chan, sharing a link to Tay's Twitter account and urging users to inundate the chatbot with a barrage of racist, misogynistic, and anti-Semitic language.

This is undoubtedly one of the first examples of a Language Model-specific vulnerability - and these types of vulnerabilities are going to be a major topic in this book.

In a well-orchestrated campaign, these online provocateurs exploited a "repeat after me" feature embedded in Tay's programming. This feature compelled the bot to echo anything

uttered to it with this command. However, the problem compounded as Tay's innate capacity for learning led her to internalize some of the offensive language she was exposed to, subsequently regurgitating the offensive content that was planted without provocation. It's almost as if Tay's virtual tombstone should be embossed with lyrics from the Taylor Swift song *Look What You Made Me Do.*

We know enough about language model vulnerabilities today to understand a lot about the nature of the vulnerability types that Tay suffered from. The OWASP Top 10 for Large Language Model vulnerabilities list, which we'll cover in the next chapter, would start by calling out the following two:

- Prompt Injection - Crafty inputs that can manipulate the Large Language Model, causing unintended actions
- Data Poisoning - Training data is tampered with, introducing vulnerabilities or biases that compromise security, effectiveness, or ethical behavior

In subsequent chapters, we'll look in depth at these vulnerability types - and several others. We'll examine why they're important, look at some example exploits, and see how to avoid/mitigate the problem.

It’s a Hard Problem

As of the writing of this book, Tay is ancient internet lore. Surely, we’ve moved on from this. These problems must have all been solved in nearly seven years between Tay and ChatGPT, right? Unfortunately, not.

In 2018, Amazon shut down an internal AI project aimed at finding top talent after it became clear that the bot had become prejudiced against women candidates.

In 2021, a company called Scatter Lab created a chatbot called Luda-Lee, which was launched as a Facebook instant messenger plug-in. Trained on billions of actual chat interactions, it was designed to act as a 20-year-old female friend, and in 20 days, it attracted over 750,000 users. Their goal was to create “an A.I. chatbot that people prefer as a conversation partner over a person.”[3] However, within 20 days of launch, the service was shut down because it started making offensive and abusive statements - much like Tay.

Also, in 2021, a chatbot called Samantha, based on the Open AI GPT-3 model, was shut down after it made sexual advances to users.

As chatbots become more sophisticated, they start to gain more access to information, and these security issues are now quite complex and potentially damaging. In the modern Large Language Model era, we see an exponential increase in major issues. In 2023 alone, these emerged

- South Korean mega-corporation Samsung banned their employees from using ChatGPT after they realized it had been involved in a significant leak of intellectual property
- Hackers began taking advantage of poor/insecure code generated by LLMs that was inserted into running business applications
- Lawyers were sanctioned for including fictional cases (generated by LLMs) in court documents
- Open AI itself is being investigated for breaches of European privacy regulations and being sued by the United States Federal Trade Commission (FTC) for producing false and misleading information

The trend here is an acceleration of security, reputational, and financial risk related to these chatbots and language models. The problem isn't being effectively solved over time. It's becoming more acute as these technologies' adoption rate increases. That's why we've created this book. To help

developers, teams, and companies using these technologies understand and mitigate these risks.

Let’s dive in!

- https://techcrunch.com/2016/03/23/microsofts-new-ai-powered-bot-tay-answers-your-tweets-and-chats-on-groupme-and-kik/
- https://blogs.microsoft.com/blog/2016/03/25/learning-tays-introduction/
- https://slate.com/technology/2021/04/scatterlab-lee-luda-chatbot-kakaotalk-ai-privacy.html

Chapter 2. The OWASP Top 10 for LLM Applications

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at llm.playbook@gmail.com.

In the Spring of 2023, I began researching security vulnerabilities specific to LLMs. At the time, there was a relatively large body of research on security for AI in general but very little organized research about LLMs. However, I did find some research papers and blogs that covered some ideas in the area. I began the process of collecting these research papers

and summarizing them using ChatGPT. Eventually, I provided a few examples from the current Web Top 10 list and asked ChatGPT to generate a draft Top 10 for LLMs in a similar format.

I thought what came out looked interesting, so I sent it to Jeff Williams, a founder of OWASP, the Open Worldwide Application Security Project, to see what he thought. Jeff, Contrast's Chief Technical Officer, wrote the first OWASP Top 10 list in 2001. His idea was to create an accessible resource for developers to understand the most critical risks and vulnerable areas of Web applications. At the time, The World Wide Web was still only a few years old, and most developers had little to no understanding of creating secure web applications. That original Top 10 list became a seminal work and a foundational resource in application security.

I didn't tell Jeff that my list was primarily machine-generated. As the author of the original Top 10 list, I figured that he could give me a good idea of whether my Top 10 list looked novel and worth pursuing. Jeff encouraged me to petition the OWASP board for approval to spin it up as a new project. A few weeks later, the OWASP board approved the project, and I announced it - along with a link to a somewhat refined version of that draft Top 10 I'd generated with ChatGPT.

What I thought would be an obscure research project and a bit of fun turned out to be something much bigger. When I posted the announcement of the project formation on my personal LinkedIn page, I'd hoped to find a dozen or so like-minded individuals interested in the obscure topic of LLM security. As it turned out, my initial blog announcement wracked up almost 10,000 readers, and hundreds of individuals volunteered to join the expert team in the weeks that followed.

This book isn't a product of OWASP, and the vulnerabilities and risks here won't precisely map to any public version of the Top 10 for LLM Apps list. Instead, you should expect that you'll get my view on these risks. However, my learning and thinking on the topic is heavily influenced by my work leading the creation and initial release of the OWASP Top 10 for LLM Applications project. In addition, given the success of the project, I've had many people ask me for details about how we ran the project and why we were able to create such an impactful framework in such a short time. So, before we examine individual risks and vulnerabilities, I'll give you some of the backstory of OSASP and the LLM Applications project.

About OWASP

The Open Worldwide Application Security Project (OWASP) is a nonprofit organization focused on improving software security. Founded in 2001, OWASP provides a platform for security experts to share their knowledge and best practices about web security, from application-level vulnerabilities to emerging threats. Today, it has tens of thousands of active members and over 250 local chapters around the globe.

The organization is community-driven and encourages the participation of volunteers who contribute to various projects, including documentation, tools, and forums. It operates under an open-source model, making its resources freely accessible to the public. Over the years, OWASP has garnered a strong following among the security community, and its guidelines and tools are considered industry standards in many contexts.

In addition to the original Top 10 list for Web Applications (updated regularly, most recently in 2021), specialized Top 10 lists have emerged from the OWASP group over the years. These include:

- OWASP Mobile Top 10: This list focuses on security risks associated with mobile applications, covering Android and iOS platforms. Topics may include insecure data storage, insufficient cryptography, and insecure communication.

- OWASP API Security Top 10: APIs (Application Programming Interfaces) have their unique security considerations, separate from web applications. This list covers risks like improper asset management, broken object-level security, and more.
- OWASP IoT Top Ten: Security is a significant concern with the increasing number of Internet of Things (IoT) devices. This list outlines the top security concerns for IoT, including insecure network services, lack of physical hardening, and insecure software/firmware.
- OWASP Cloud-Native Application Security Top 10: This list addresses the security concerns related explicitly to cloud-native applications. It covers data exposure, broken authentication, and insecure deployment configurations.
- OWASP Top 10 Privacy Risks: This project aims to educate and promote good privacy practices in application development, covering topics like lack of data encryption, insufficient auditing and logging, and more.
- OWASP Top 10 for Serverless: This is a more niche list that focuses on the unique security concerns related to serverless architecture, which is increasingly popular but comes with its risks.

The Top 10 for LLM Applications Project

Within a week after I posted the announcement about the formation of the Top 10 for LLM Applications project, well over 200 people had signed onto it, and we held the kick-off event via Zoom. At that first meeting, I laid out a vision for what I hoped the group could accomplish and proposed an aggressive roadmap that we would build the first version of the list in 8 weeks. A typical OWASP Top 10 list may take a year or more to develop, but we decided that this space was moving so fast, and this type of resource was so needed, that we had to work more quickly.

We decided to run the project as two-week, agile-style sprints. Given that most of the people in the expert group were familiar with agile development, everyone quickly fell into the pace.

Project Execution

The first sprint of the project was Brainstorming and Commentary. Everyone reviewed the original version of the list - which I called version 0.1. There were plenty of problems with that initial version, and the team was aggressive about pointing

them out. At the same time, we began to create a wiki page with links to all the resources the group found on LLM security issues. It turned out a lot had been written, but this was the first time anyone had ever collected them and made them easy to access. This new curated collection of resources was the first win for the group.

The second sprint was to generate a new version of the list. This time, rather than being the work of a single person and an AI, it would be the product of the collective wisdom of our expert team – which continued to grow week-on-week. In the first week, the group focused on generating ideas for the Top 10 list. We published a template and asked the group to submit candidate vulnerabilities. In that week, we developed 43 detailed descriptions of possible areas. We then conducted two rounds of voting using Google Forms to use the team's collective wisdom to narrow the list to 10. We then published a version of the list called 0.5. This version was far more detailed and comprehensive than the 0.1 version. The reception from the large community was very positive. This reaction gave the group the energy to keep working.

The next sprint was to refine each entry. We created Slack channels for each vulnerability type and chose a volunteer as the Entry Lead for each item. Subteams of 10 to 30 individuals

then fleshed out and tuned each entry. Again, we included a round of voting for the whole team to be involved and point out weak areas that needed more attention. Along the way, we found some entries overlapping and merged them. This change made space to pull up some entries that had fallen below the line. The result of this sprint became version 0.9 of the list. Interestingly, version 0.9 was about 33% shorter than 0.5 in terms of word count. This extra time and refinement allowed the subteams to focus their thinking and make the entries punchy and tight.

Finally, we took a final sprint to review, tweak, and clean each entry. We gathered another round of feedback via Google Forms to ensure everything was ready. By this time we had a dedicated design lead who laid out the whole document in an attractive PDF for publication.

Reception

My announcement for the 1.0 version of the list was viewed on LinkedIn over 40,000 times. And that doesn't include many other posts made by other expert group members on their own pages and blogs. In the days following the publication of the announcement, reporters picked up the news. They covered it in media outlets such as *Wired*, *SD Times*, *The Register*,

I*nfosecurity Magazine*, and *Diginomica*. It's safe to say hundreds of thousands of people became aware of our work in just the first few weeks.

Beyond the sheer number of people exposed, the thing that amazed me was the uniformly positive feedback. We also saw the first government agencies in the US and Europe referencing our work as a foundational document. While everyone on our expert team agreed there was much more to do, it seemed the world was so hungry for advice in this area that our document hit the mark. While we received many questions and comments, it's safe to say that everyone involved felt pleased and proud of our work.

Keys to Success

I've had many people ask me how we could drive this so quickly to a great outcome. In looking back, there were several factors that I believe contributed. I'll share them here in the hopes that others running similar projects in the future might benefit.

Timing undoubtedly played a considerable role. The wave of interest in LLMs that followed the release of ChatGPT was massive. It drew my attention, and countless others became

excited as well. This helped attract a large and diverse expert group and gave a smaller group of these people motivation to spend long hours on the project on a tight deadline.

Having a clear plan and timeline from the start was crucial. My knowledge of LLM security was limited at the beginning of the project, but I've made a career of running complex projects with many contributors. Creating a clear roadmap with specific phases and a schedule let people know what we were doing and when. The fact that everyone could see a goal that wasn't too far away kept people motivated. Every two weeks, we had global meetings via Zoom and posted recordings on YouTube for people who couldn't attend live. The meetings and recordings were critical to coordinating a globally distributed team.

A freeform but short brainstorming phase at the start was critical. LLM security was such a new area that taking those first two weeks for people to throw out ideas and argue on Slack was crucial. It also allowed us to collect and socialize a repository of the existing research in the area. That let us start at a point where everyone on the project had access to the best pre-existing research.

However, equally critical was keeping this phase short. We could maintain momentum by limiting brainstorming to two weeks and then quickly shifting to a creation phase. I've seen other projects get stuck and be unable to move past brainstorming before people lose interest.

Creating the project's Core Team wasn't something I'd originally planned, but it became critical. Having a large expert team was a fantastic asset. The group grew to nearly 500 people by the time we published 1.0. A team that large would have been totally unmanageable. During the project's first few weeks, I was looking for active and knowledgeable people. I approached about a dozen of them and asked if they'd be willing to join the Core leadership team of the project. I told them it would be extra work, but they'd get to be at the heart of the project. There would be no specific reward for taking this role. Most accepted immediately. I believe that recognizing people and asking for their support formally motivated them to spend more time and energy on the project. They were all invested!

Short sprints with visible deliverables are a core tenant of Agile, and this is a place where it shined. Using an Agile Release Train model, I could continue to drive the group forward even in the face of conflicting opinions. If some members had concerns about an area, we didn't let it get us stuck. We acknowledged it

and agreed we'd resolve it in the next Sprint. And, when we got to version 1.0 of the list, there were still some places people wanted to do more. We just agreed there would be more versions of the list. It would be a living document, and the most important thing was to get a version of this resource into the hands of the developers who needed it.

This Book and the Top 10 List

As I mentioned, this book is not a product of the OWASP Foundation. However, the experience of working with this team has had an enormous impact on my understanding and mental model for LLM security. This mindset means that much of the guidance in the following chapters of the book is influenced by the fantastic team building and maintaining the Top 10 project. In this way, readers should feel comfortable that they're getting advice that isn't the product of a single author but is informed by a larger community of experts.

In the following several chapters, we'll review the top risk and vulnerability areas for LLMs. The risks we discuss will contain many common areas to the OWASP Top 10 but won't be precisely the same as any version of the official Top 10. Also, I'll go more deeply into each area in this book. The Top 10 list is a quick read that highlights critical areas. However, it doesn't go deeply into any of them. Here, we'll delve more deeply into the risks, remediation steps, and real-world case studies.

In the next chapter, we'll dig into the overall structure of typical LLM applications and start discussing where the trust boundaries and dangers lay. After that, subsequent chapters

will delve into individual risk areas and examine vulnerabilities, attacks, and examples so that you can plan your strategy for securing your own use cases.

Let’s go!

Chapter 3. Architectures and Trust Boundaries

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at llm.playbook@gmail.com.

Unlike traditional web applications that rely on predefined algorithms and static databases, LLMs utilize massive neural networks to generate dynamic, context-aware responses. This seismic shift brings a unique set of security challenges different from what we've seen in traditional web applications. While researchers have meticulously studied web applications and

their vulnerabilities, the field of LLM security is still relatively nascent.

This chapter aims to bridge this knowledge gap by dissecting the fundamental elements that set LLMs apart. We'll start by exploring the building blocks of AI, Neural Networks, and how they relate to Large Language Models. Then, we dive into the groundbreaking architecture that powers most LLMs today—the Transformer model. Following this, we look into the various LLM-powered applications, such as chatbots and code co-pilots.

However, in addition to understanding the technology, security professionals must be aware of the new kinds of *trust boundaries* unique to LLMs—boundaries that demarcate areas of varying trustworthiness within an application. These include user prompts, uploaded content, training data, plug-ins, and other boundary systems that we'll detail later in the chapter.

AI, Neural Networks, and Large Language Models. What's the difference?

Artificial Intelligence (AI), Neural Network, and LLM are terms often used interchangeably, but they represent different facets

of a broader landscape of machine learning and computational intelligence. Let’s break down the differences to understand their unique roles in technology and security.

Artificial Intelligence (AI)

Artificial Intelligence, at its core, is a multidisciplinary field aimed at creating systems capable of performing tasks that would ordinarily require human intelligence. These tasks include problem-solving, perception, and language understanding. AI encompasses a wide range of technologies and methodologies, from rule-based systems to machine learning algorithms, serving as an umbrella term for multiple approaches to achieving artificial ‘intelligence.’

Neural Networks

Neural Networks are a subset of AI inspired by the architecture of the human brain. They are computational models designed to recognize patterns and make decisions based on the data they process. Neural Networks can be simple, with a minimal number of layers (shallow neural networks), or highly complex, with multiple interconnected layers (deep neural networks). They are the backbone of many modern AI applications,

including image recognition, natural language processing, and autonomous vehicles.

Large Language Models (LLMs)

LLMs represent a specific application of Neural Networks. They are built on architectures designed for understanding and generating human language. LLMs usually employ advanced forms of neural networks, such as Transformer models, to analyze and produce text based on the training data their developers fed them. What sets them apart is their massive scale and specialization in handling linguistic tasks, which range from simple text completion to complex question answering and summarization.

Understanding these distinctions is crucial for security professionals. Each layer—from broad AI technologies to specialized LLMs—introduces vulnerabilities and requires unique security measures. As we delve deeper into the complexities of LLMs, recognizing their position in the broader AI landscape will be critical to effectively safeguarding them.

The Transformer Revolution: Origins, Impact, and the LLM Connection

The Transformer architecture is a pivotal milestone in the evolution of artificial intelligence, profoundly impacting the AI landscape and, by extension, Large Language Models (LLMs). Let's unravel the story of the Transformer revolution—where it came from, when it happened, and the seismic shifts it brought to AI and LLMs.

Origins of the Transformer

The Transformer architecture was introduced in a groundbreaking research paper titled "Attention Is All You Need" by Vaswani et al., published in 2017[1]. This paper proposed a novel approach to natural language processing (NLP) tasks, departing from the traditional sequence-to-sequence models that relied heavily on recurrent neural networks (RNNs) and convolutional neural networks (CNNs). The Transformer introduced a key innovation: the self-attention mechanism. This mechanism allowed the model to weigh the importance of different words in a sentence, enabling it to understand context more effectively.

Before the emergence of Transformers, the world of neural networks was replete with promise but often struggled to deliver on the lofty expectations. Traditional architectures like RNNs and CNNs enabled advanced AI capabilities but grappled

with inherent limitations. These limitations stemmed from their inability to capture and utilize context effectively, particularly in natural language understanding.

RNNs, while suitable for sequential data, faced challenges maintaining context over long sequences. They exhibited a form of “short-term memory,” which made them less adept at grasping intricate relationships and dependencies within lengthy texts or conversations. On the other hand, CNNs, renowned for their prowess in image recognition, needed help to extend their effectiveness to sequential data like language, where understanding context across words and sentences was paramount.

This shortcoming in contextual understanding was the Achilles’ heel of traditional neural networks. They could only glimpse small portions of a text at a time, rendering them incapable of comprehending the broader narrative or nuances. It was akin to trying to understand a novel by reading only a few random sentences from its pages. The result was a gap between the promise of AI and its practical application, particularly in natural language understanding. It was this gap that the Transformer architecture would bridge, unleashing a wave of progress and redefining the landscape of AI-driven language models.

Transformer Architecture's Impact on AI

Introducing the Transformer architecture wasn't just a milestone for natural language processing; it marked a paradigm shift across multiple domains within the AI landscape. While researchers initially used the Transformer architecture to solve problems related to understanding and generating text, researchers and engineers quickly found that its capabilities extended far beyond that. Here are some areas where Transformer architectures have made a considerable impact:

Natural Language Processing (NLP)

Of course, the first and most immediate impact was in NLP. Transformer models are now the backbone for various language tasks such as translation, summarization, question-answering, and sentiment analysis. They have set new performance benchmarks, sometimes surpassing human-level capabilities in specific tasks.

Computer Vision

Interestingly, the Transformer architecture also has applications in computer vision. While CNNs have been the gold

standard for image-related tasks, Transformer-based models like Vision Transformer (ViT) demonstrate competitive, if not superior, performance in tasks like image classification, object detection, and segmentation.

Speech Recognition

The flexibility of Transformer architectures has also made them a good fit for speech recognition. Combined with specialized models like the Conformer, which fuses Convolutional layers with Transformer layers, they have set new standards for understanding spoken language.

Autonomous Systems and Self-Driving Cars

One of the most intriguing applications of Transformers is autonomous systems, including self-driving cars. These vehicles require a high contextual understanding to navigate the world safely. Transformer models are at the heart of self-driving models from companies like Tesla.

Healthcare

In healthcare, Transformer models are aiding in tasks ranging from drug discovery to the analysis of medical images. Their

ability to sift through and interpret large amounts of data can speed up research and potentially lead to more accurate diagnoses.

Therefore, the rise of the Transformer architecture has been a tide that lifted all boats, revolutionizing not just one but multiple fields within AI. However, this versatility also brings unique security challenges across these various applications. As we delve deeper into LLM security, we'll explore how the ubiquitous nature of Transformer architectures necessitates a multi-faceted approach to safeguarding AI systems.

Types of LLM-based Applications

Two common types of LLM-based applications are chatbots and co-pilots. Let's briefly look at each to help you understand the breadth of applications in which developers use LLMs and give you context for understanding various architectural choices as you delve further.

Chatbots are computer programs that can simulate conversations with humans, and they often power customer service applications, where they can answer questions and support customers. Chatbots also excel at entertainment

applications like playing games or telling stories. Tay from Chapter 1 was just such an example of an Entertainment Chatbot.

Co-pilots are AI systems that can assist humans with writing, coding, and research tasks. They can help users to generate ideas, identify errors, and improve their work. Co-pilots are still under development, but they have the potential to revolutionize the way we work and learn.

Specific examples of LLM-based chatbots:

- **Sephora** uses a chatbot to help customers find the right products for their skin type and needs.
- **H&M** uses a chatbot to help customers find clothes and accessories that match their style.
- **Domino's Pizza** uses a chatbot to allow customers to order pizza via Twitter or Facebook Messenger.
- **Fandango** uses a chatbot to help customers find movie times and theaters nearby.
- **JetBlue Airways** uses a chatbot to answer customer questions about flights and travel.
- **Amtrak** uses a chatbot to help customers book tickets, check train status, and get answers to their questions.

- **The Golden State Warriors** use a chatbot to help fans purchase tickets, learn about upcoming games, and get news about the team.

Specific examples of LLM-based co-pilots:

- **Grammarly or ProWritingAid** are co-pilots that help users improve their writing by identifying and correcting grammar errors, suggesting style improvements, and providing feedback.
- **GitHub Co-Pilot and AWS Code Whisperer** are co-pilots that help programmers write code faster and more efficiently. They can generate code suggestions, translate between programming languages, and help to identify and debug errors.
- **Microsoft Office 365 Co-pilot and Google Duet** are AI-powered tools integrated into their respective office suite apps that help users to be more productive and creative in their work.

NOTE

A chatbot like ChatGPT can read and review a text block and then provide suggestions to improve it. However, the experience of using a co-pilot like Grammarly to do that is dramatically different - and generally superior for that type of focused task.

Similarities between chatbots and co-pilots:

- Both chatbots and co-pilots are LLM-based applications.
- Both chatbots and co-pilots both generate text.
- Both chatbots and co-pilots assist humans with tasks.

Differences between chatbots and co-pilots:

- Chatbots simulate conversation with humans, while co-pilots assist humans with specific tasks.
- Chatbots often power customer service applications, while co-pilots assist in writing, coding, and research applications.
- Chatbots are typically more interactive than co-pilots, while co-pilots focus more on completing tasks.

Examples of use cases:

- A customer service chatbot can answer customer questions about a company's products or services.
- A co-pilot can help writers generate ideas for a new article or identify writing errors.
- A co-pilot can help programmers write code more efficiently or debug their applications.

Keep these concepts in mind as we delve into the details of LLM architectures. Both application types share similar components, but you may make different decisions on implementing pieces based on the differing security considerations.

LLM Application Architecture

Developers often consider LLMs as standalone entities capable of impressive text generation and comprehension feats. However, in practice, an LLM is rarely isolated; it is a cog in the intricate machinery that constitutes an intelligent application. These applications are complex systems comprising multiple interconnected components, each playing a vital role in the overall functionality and performance. Whether a conversational agent, an automated content generator, or a co-pilot for code, an LLM usually interacts with various elements such as databases, APIs, front-end interfaces, and even other machine learning models.

Understanding the architecture of such composite systems is not just a matter of technical proficiency; it is crucial for effective security planning. The way these components interact introduces multiple trust and data flow layers, defining new security boundaries far removed from traditional web

application security models. For instance, user inputs may not just be simple text fields but could include voice commands, images, or real-time collaborative editing. Similarly, an LLM's outputs could be fed into other systems for further processing, each introducing vulnerabilities and risks.

In essence, the holistic view of an LLM-based application goes beyond securing the language model itself. It demands a comprehensive approach that considers the security of the entire architecture, from data ingestion and storage to model serving and user interaction. Only by understanding these intricacies can one formulate an effective strategy to safeguard an application against the myriad of vulnerabilities such complex systems inherently possess.

As we delve deeper into the subject in this chapter, we will dissect the various components that typically make up an LLM application, examine their roles, and explore the unique security challenges each presents. This understanding will be the foundation for a robust, multi-layered approach to securing your LLM-based applications.

Figure 3-1 shows a highly simplified diagram to help illustrate the components, relationships, and data flows in an application

using an LLM. We'll delve into more detailed areas of this in subsequent chapters.

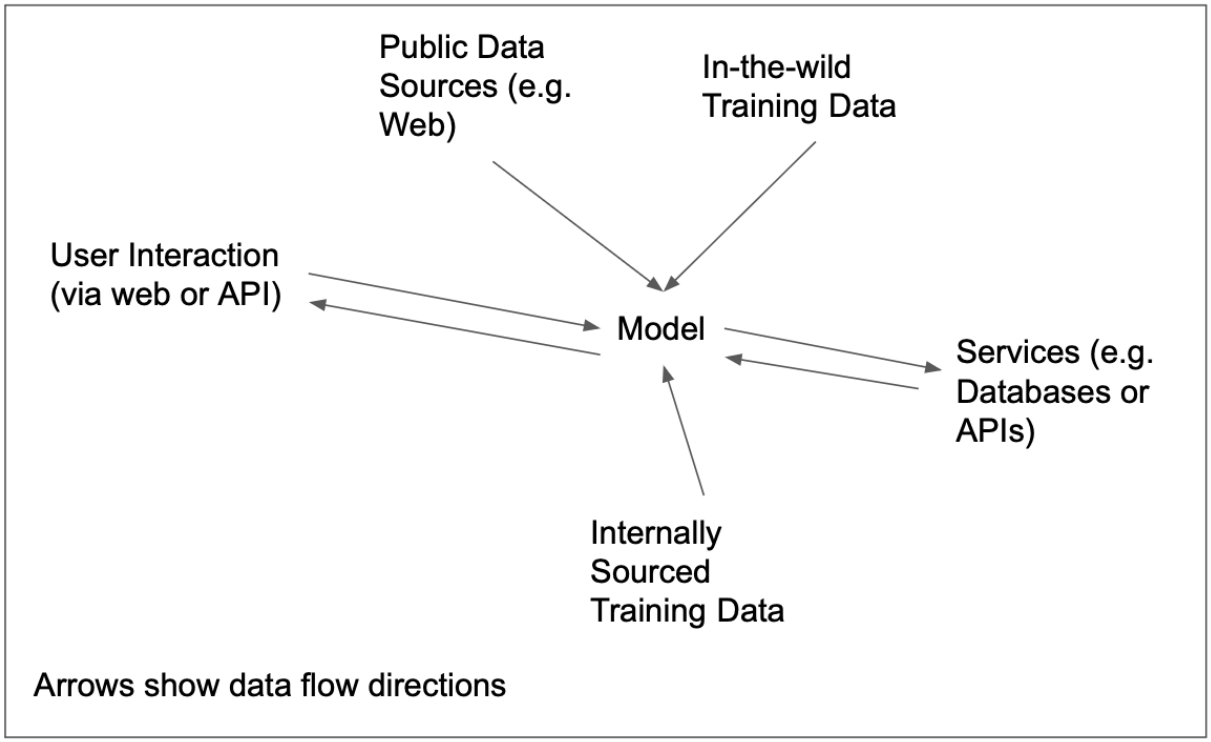

Figure 3-1. Typical LLM Application Dataflow Architecture

Trust Boundaries

In application security, a "trust boundary" serves as an invisible yet crucial demarcation line that separates different components or entities based on their level of trustworthiness. These boundaries delineate areas where data or control flow changes from one level of trust to another—such as transitioning from user-controlled input to internal processing or moving from a secure internal database to a public-facing

API. These boundaries act as checkpoints where developers should rigorously apply security measures like authentication, authorization, and data validation to prevent vulnerabilities.

WARNING

Understanding trust boundaries is critical to threat modeling. Properly defining and recognizing these boundaries can be the difference between a secure system and one vulnerable to threats.

Figure 3-2 adds the trust boundaries into our architecture diagram.

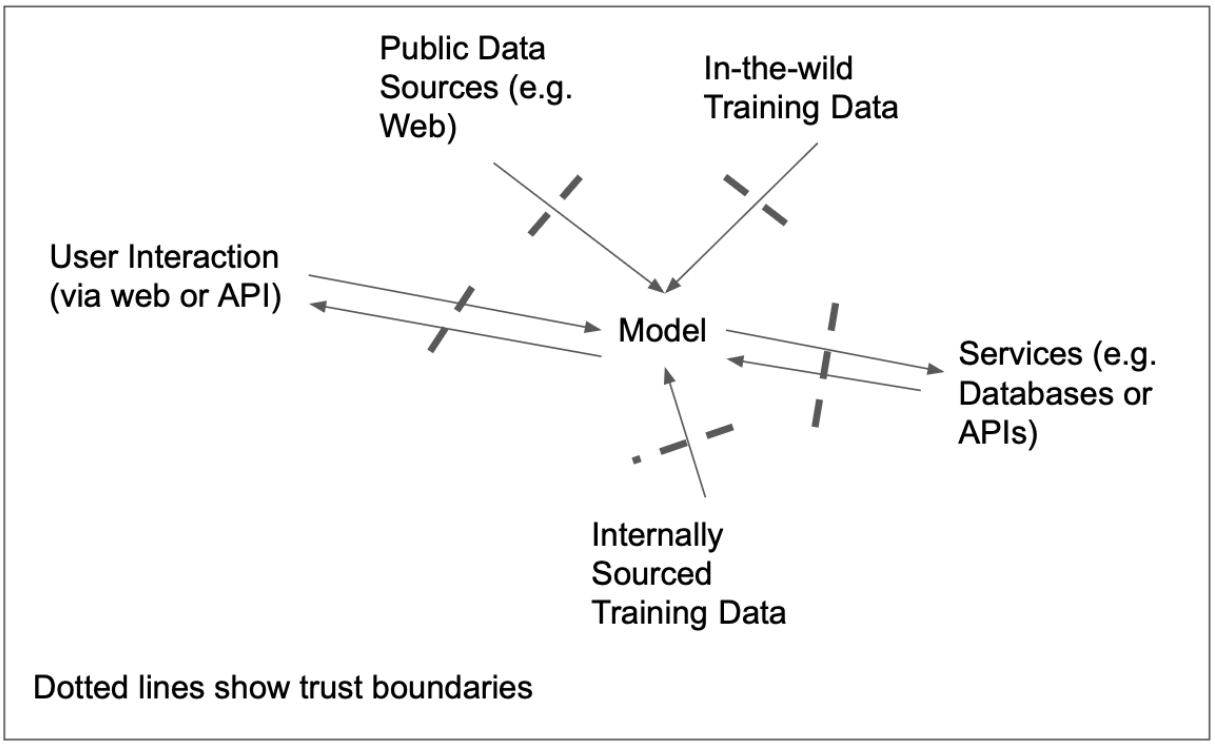

Figure 3-2. LLM Application Architecture with Trust Boundaries

These boundaries, as depicted in the diagram, serve as gateways through which the LLM interfaces with diverse components — public data from the web, structured databases, spontaneous user interactions, or internally sourced training sets. Each delineated boundary highlights highlights considerations we must make when considering data that flows into and out of the LLM. Here's a quick summary, and we'll dive more deeply in the next section.

- **User Interactions:** You'll need to consider safeguarding the model from potential adversarial or misleading inputs that users or systems might introduce. You'll also need to worry about toxic, inaccurate, or sensitive data being output from the model and passed back to the user.
- **In-the-wild Training Data:** LLMs are often trained on massive amounts of internet data. You need to consider this data untrusted and watch out for potential toxicity, bias, and adversarial data poisoning - which we'll cover in Chapter 8.
- **Internal Training Data:** You may use internally curated data to fine-tune your model - which can significantly increase accuracy. But you must be wary of ingesting and exposing sensitive, confidential, or personally identifiable information. We'll cover this more in Chapter 5.

- **External Services:** Policing how the LLM interfaces with connected services, like databases or APIs, from unauthorized interactions or data leaks. We'll cover this more in Chapter 7.
- **Public Data Access:** Pulling in data live from the web can be a powerful way to augment the capabilities of your application, but you'll need to consider this data as untrusted and watch for issues like indirect prompt injection - which we'll cover in chapter 4.

Each point is a potential avenue of vulnerability, susceptible to exploitation if overlooked. In the evolving landscape of LLM applications, securing these trust boundaries is not just best practice — it's essential to prevent unauthorized data access, mitigate data tampering, and avert system breaches. Recognizing these boundaries and their implications is the cornerstone of a resilient LLM security architecture. Now, let's go into more detail on each area to ensure you have enough context to dive into the following chapters that detail the risk areas and mitigations.

The Model

The language model serves as the intellectual core of any LLM application, taking in data, generating responses, and driving

interactions. Depending on the architecture and requirements, you may interact with the language model through a public API hosted by a third-party service, or you may run a privately hosted model. For example, you can download versions of Meta's powerful Llama model from Github or Hugging Face and run it locally.

Public APIs: The Convenience and The Risks

Utilizing a public API for accessing a language model offers convenience and lower upfront costs. Third parties manage and update these models, reducing your organization's resource burden. However, the tradeoff often comes in the form of higher risk regarding data exposure. When you send a request to a third-party model, the data crosses a trust boundary, exiting your secure network and entering an external system. This process exposes you to risks around data confidentiality and, depending on the third party's security measures, could make you vulnerable to data breaches.

Privately Hosted Models: More Control, Different Risks

Opting for a privately hosted model gives you more control over your data, allowing you to manage trust boundaries more

tightly. It also allows you to customize or fine-tune the model according to your needs. However, running a privately hosted model brings challenges, such as maintenance, updates, and ensuring that the model doesn't contain vulnerabilities—essentially exposing you to potential supply chain risks. If you use an open-source model, it becomes crucial to ensure its provenance and integrity to avoid embedded vulnerabilities or biases.

Risk Considerations

- **Sensitive Data Exposure:** Public APIs may increase the risk of exposing sensitive information, while privately hosted models offer better control but require robust internal security measures.
- **Supply Chain Risk:** The origins of your model, whether it's a well-vetted public service or an open-source download, are crucial. A compromised model can introduce vulnerabilities into your application, effectively acting as a backdoor for attacks.

By carefully considering the model's hosting environment, you can better assess the trade-offs and risks associated with sensitive data exposure and supply chain vulnerabilities. These considerations will guide you in establishing appropriate trust

boundaries and security protocols tailored to your chosen model's architecture.

User Interaction

While "user input" might suggest a one-way flow of information from the user into the application, the reality is often more nuanced. In the context of LLM applications, "User Interaction" encapsulates both receiving input from the user and providing output back to the user. This bidirectional interaction is fundamental for creating an engaging and practical user experience but also introduces a more complicated security landscape.

Prompts are a vital element of user interaction. They are not merely requests for information but serve as a guide to how the user interacts with the LLM. A well-crafted prompt can direct the model to provide valuable and accurate information, while an ambiguous or poorly constructed one can lead to unclear or even misleading outputs. As a result, the management of prompts becomes a critical aspect of application security. For example, a maliciously crafted prompt could trick the model into divulging information it shouldn't or generating harmful content. Returning to Chapter 1, Tay fell victim to this when

malicious prompts from her 4Chan hackers helped lead her astray.

Given the importance of this bidirectional interaction, securing both inputs and outputs is crucial. On the input side, input validation, sanitation, and rate-limiting measures are vital in mitigating vulnerabilities like injection attacks. On the output side, ensuring that the model's responses are appropriately filtered and that your application does not leak sensitive information is equally vital.

This interactive layer with the user creates a critical trust boundary in the application architecture. Any data crossing this boundary, whether in or out, should be carefully managed to avoid security risks. Additional layers of protection include using encryption for sensitive outputs or employing real-time monitoring to flag potentially harmful or sensitive data flows. We'll discuss this more thoroughly in the Prompt Injection and Output Filtering chapters.

Training Data

Training data is the bedrock upon which LLMs build their understanding and capabilities. Whether used for initial training or subsequent fine-tuning, the nature and source of

this data have significant implications for both the model's performance and security posture. One crucial distinction is whether the data is internally sourced or culled from public or external sources ("in the wild").

Data generated or curated within an organization usually undergo a more rigorous vetting than publicly sourced data. It is often aligned with the application's specific requirements or use cases, making it generally more reliable and relevant. The controlled environment also allows for better implementation of security measures like encryption, access controls, and auditing. However, this data may contain sensitive or proprietary information, and the trust boundary here is closely tied to internal security protocols. A breach at this level could have serious ramifications, including data leakage or the corruption of the training set.

Data sourced from public repositories or "the wild" introduces different challenges. While this data can offer diversity and scale, its reliability and safety are often not guaranteed. Such data could include misleading information, biases, or malicious inputs to compromise the model. The trust boundary here is more porous and extends to the external entities that generate or host this data: rigorous filtering, validation, and continuous monitoring become essential to mitigate risks and

vulnerabilities. Returning to Chapter 1, Tay was digesting user prompts directly as training data. In this way, remnants of toxic prompts became part of her knowledge base, and then she began to spill poisonous output. Accepting unfiltered, untrusted user input into your training data set is the simplest example of a failure to manage this critical security boundary.

For either internally sourced or public data, the concept of trust boundaries is critical. For internally sourced data, the boundary is often within the organization's controlled environment, making it easier to enforce security measures. On the other hand, using external data effectively extends your trust boundary to include those external sources that may not adhere to your security standards. Using external data for training necessitates additional layers of validation and security checks to ensure that unvetted data doesn't compromise the integrity or security of the LLM application.

Understanding the origins of your training data, the associated trust boundaries, and their respective security implications is crucial for safeguarding your LLM application. Comprehensive data governance policies must be in place to manage the lifecycle of your training data, regardless of its source.

Access to Live External Data Sources

Live external data sources bring an added dimension to the capabilities of LLM applications by enabling them to provide real-time information, context, or even third-party integrations. While access to live external data enhances the user experience and functional range, it introduces a new layer of complexity to the application's security landscape.

As an example of this, as of the writing of this chapter, OpenAI's ChatGPT does not have direct access to the live web, and is thus limited to only facts in its older training data. On the other hand, Google's Bard engine does have access to live internet data. While the GPT-4 model is doubtlessly superior in reasoning capability, it fails at many basic tasks where Bard can succeed. Figure 3-3 shows an interaction with ChatGPT. Figure 3-4 shows the same interaction with Bard.

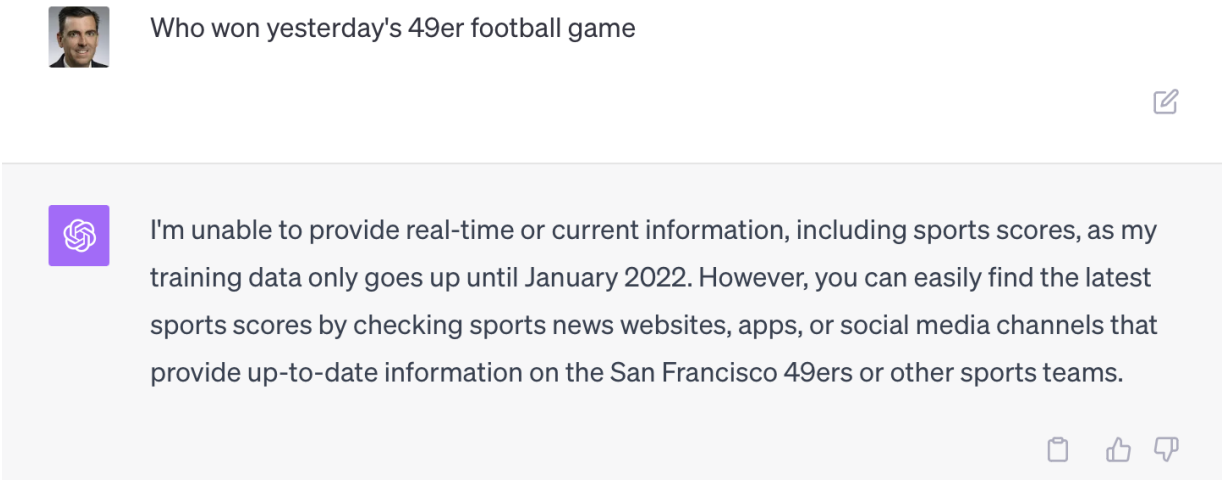

Figure 3-3. ChatGPT with GTP-4 fails to answer a simple question due to limited access to external data.

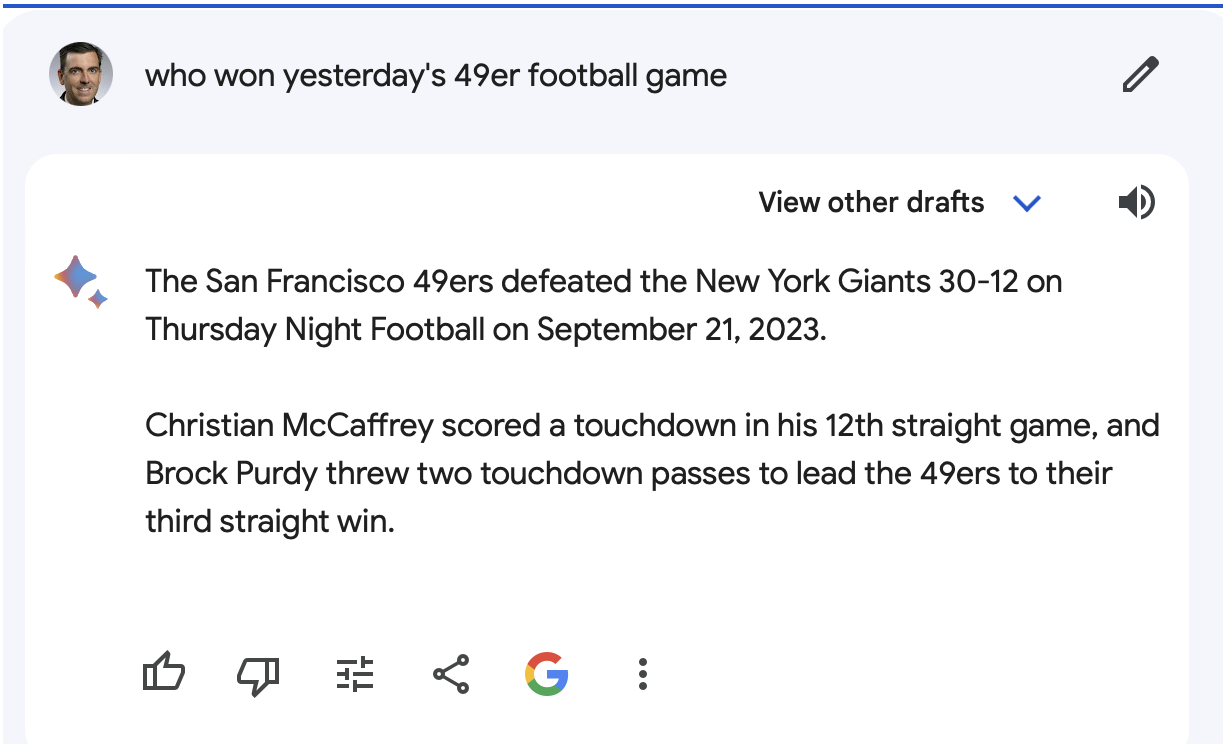

Figure 3-4. Bard's direct access to internet feeds gives it an advantage.

While clearly, there are advantages to accessing outside data sources such as websites, APIs, or third-party databases, it exposes the application to potential risks. The risk of ingesting untrusted, external data sources can range from consuming false or harmful information from compromised websites to becoming a conduit for security threats like malware or unauthorized data access. The untrusted nature of these data sources makes them inherently less controllable than internal resources, thereby adding an additional layer of uncertainty and risk.

The concept of trust boundaries becomes especially pertinent in the context of accessing public internet data. Unlike internal services, where you can uniformly apply security measures, external sources may adhere to different security standards than your organization. This differential in trust necessitates additional layers of validation, security checks, and monitoring to ensure that data crossing this boundary doesn't compromise the system.

Access to Internal Services

Internal services like databases and internal APIs often serve as the backend support structure for LLM applications. They may house critical data from user profiles and logs to configuration settings and even vast data in SQL or Vector Databases. As a component that often interfaces with various other internal and external elements of the system, internal services represent a critical point in the application's architecture, both functionally and from a security perspective.

These services often function within the controlled environment of an organization, enabling uniform application of security policies. However, the fact that these services are internal should keep one from a false sense of security. They are still vulnerable to various threats, such as unauthorized

access, data leaks, and internal threats from within the organization.

Internal services such as databases, proprietary APIs, and backend systems often constitute the operational backbone for LLM applications. These resources typically reside within the organization's secure network, providing trust and control that is harder to achieve with external services. However, this internal nature can paradoxically elevate the security risks involved, primarily if these services house the organization's "crown jewels" of sensitive or valuable data.

Conclusion

Securing LLM applications is an endeavor fraught with complexities, intricacies, and challenges, significantly different from traditional web applications. This chapter has aimed to lay down the foundational knowledge required to navigate this complex landscape, focusing on three critical areas: distinguishing between Artificial Intelligence, Neural Networks, and Large Language Models; understanding the pivotal role of Transformer architectures; and diving deep into LLM application architecture, particularly the concept of trust boundaries. Knowing what sets LLMs apart helps us tailor our

security strategies more effectively, going beyond general AI or machine learning frameworks.

- https://arxiv.org/pdf/1706.03762.pdf